Table des matières

Cours 02 — Ranger les fichiers en bronze 1
Objectifs 1
1. Ligne contre colonne 2
2. Cinq formats 2
3. Parquet, de près 3
4. La règle du bronze 4
5. Le stockage objet et le partitionnement 4
6. DuckDB sur le lac 5
7. Contrôler un bronze, en pratique 6
À retenir 6
Exercices 7
Annexe A — Le détail des étapes du labo 7

Cours 02 — Ranger les fichiers en bronze

Théorie · Cours 2/12 — Formation Data Engineering · débutant

Où vous êtes : source → ingestion → **lac** → entrepôt → restitution.

Objectifs

- Distinguer un format en lignes d'un format en colonnes.
- Comparer CSV, JSON lines, Avro, Parquet et ORC.
- Décrire l'intérieur d'un fichier Parquet.
- Appliquer la règle du bronze : déclarer les types, garder en texte ce qu'une conversion abîmerait.
- Partitionner un lac par chemin, et reconnaître le piège des petits fichiers.
- Écrire, contrôler et interroger du Parquet avec DuckDB.

© 2026 iAlexMG. Tous droits réservés.

1. Ligne contre colonne

	Stockage en lignes	Stockage en colonnes
Ce qu'il écrit à la suite	un enregistrement entier	toutes les valeurs d'une colonne
Lire une colonne sur quatre	lit les quatre	lit celle-là
Ajouter ou modifier un enregistrement	facile, au bout du fichier	coûteux : on réécrit des blocs entiers
Compression	médiocre : des valeurs de natures différentes se côtoient	forte : des valeurs semblables se suivent
Fait pour	les applications qui traitent un enregistrement à la fois : l'ERP, le portail des garanties	l'analyse, qui lit peu de colonnes sur beaucoup de lignes

- Une requête analytique lit peu de colonnes sur beaucoup de lignes. La vibration maximale du compresseur 2 ne demande que `TagName` et `Value`, sur 275 436 lignes d'historien.
- En colonnes, on ne lit que ces deux-là, et elles se compressent bien : `TagName` ne prend que 58 valeurs, qui se répètent sans cesse.
- Les bases transactionnelles, comme celle de l'ERP, restent en lignes : elles écrivent et relisent une commande à la fois.

2. Cinq formats

Format	Organisation	Schéma et types	Compression	Usage typique
CSV	lignes, texte	aucun : tout est texte	faible, ou gzip autour du fichier	l'export d'un système : l'historien, les extraits de l'ERP
JSON lines	lignes, texte, un objet par ligne	aucun de déclaré ; imbrication permise	faible, ou gzip	les événements et les API : la télémétrie
Avro	lignes, binaire	schéma embarqué, évolution prévue	par blocs	les flux d'événements, écrits un à un

© 2026 iAlexMG. Tous droits réservés.

Format	Organisation	Schéma et types	Compression	Usage typique
Parquet	colonnes, binaire	schéma et types embarqués, imbrication permise	forte, par colonne	le lac analytique : le bronze et l'argent de ce parcours
ORC	colonnes, binaire	schéma et types embarqués	forte, par colonne	l'écosystème de Hadoop et de Hive

- CSV et JSON sont universels : d'excellents formats d'échange, de mauvais formats de stockage.
- Avro écrit vite, un enregistrement à la fois, dans un flux. Parquet lit vite, une colonne à la fois, dans un lac. ORC fait le travail de Parquet là où Hive a régné ; ce parcours n'en a pas l'usage.
- Chez Boréal, les sources arrivent en CSV et en JSON lines, et le lac est en Parquet : l'historien passe de 13,7 Mo à 1,5 Mo, la télémétrie de 70,6 Mo à 10,8 Mo.

3. Parquet, de près

Niveau	Ce que c'est	Pourquoi il existe
groupe de lignes (*row group*)	un paquet de lignes ; DuckDB en met jusqu'à 122 880	l'unité qu'un moteur lit ou saute, et traite en parallèle
bloc de colonne (*column chunk*)	les valeurs d'une colonne dans un groupe	la lecture colonne par colonne
page	une tranche d'un bloc, encodée et compressée	l'unité de compression

- Le pied de page (*footer*), à la fin du fichier, décrit le tout : le nom et le type de chaque colonne, et, pour chaque bloc, le nombre de valeurs, le minimum et le maximum. Un moteur le lit avant le reste : aucun type à deviner.
- L'encodage réduit la taille selon la forme des valeurs : un dictionnaire remplace les 58 étiquettes par de petits numéros ; un codage par plages (*run-length encoding*) écrit « 192, répété 4 000 fois ».
- La compression s'applique ensuite à chaque page : snappy, rapide, le défaut de beaucoup d'outils, DuckDB compris ; zstd, un peu plus lente, plus compacte. L'historien en un seul fichier : 4,0 Mo sans compression, 1,8 Mo en snappy, 1,2 Mo en zstd. Le bronze prend zstd : il se lit bien plus souvent qu'il ne s'écrit.
- La projection ne lit que les colonnes que la requête nomme. Le prédicat poussé (*predicate pushdown*) compare le filtre au minimum et au maximum d'un groupe, et saute le groupe si aucune ligne ne peut y répondre : `Value > 20` saute un groupe dont le maximum est 12.

© 2026 iAlexMG. Tous droits réservés.

Piège — des statistiques qui ne servent à rien. Dans un fichier quotidien de l'historien, `TagName` va de `ESS.BANC.REGIME` à `UTL.SEC.POINT_ROSEE` dans le même groupe : un filtre sur une étiquette ne saute rien. Trier par étiquette à l'écriture rendrait ces statistiques utiles.

4. La règle du bronze

On déclare les types, on ne les devine pas. Et l'on ne déclare un type que si la conversion ne perd rien.

Ce que livre la source	Dans le bronze	Pourquoi
un horodatage à format fixe, sans fuseau (l'historien)	`TIMESTAMP`	rien ne se perd : le fichier ne portait pas de fuseau non plus
un horodatage avec son décalage (la télémétrie)	texte	le convertir jetterait le décalage
un nombre, vide quand la mesure manque	`DOUBLE`, le vide devenu nul	la valeur et son absence survivent
un code de qualité entier	`INTEGER`	le dictionnaire le garantit
un code, un numéro à zéros de tête, une date SAP	texte	la conversion appartient à celui qui connaît la règle métier

- Un montant se convertit en `DECIMAL`, jamais en `DOUBLE`, qui représente mal les centimes : une somme peut tomber à un cent près.
- Le bronze de ce cours range les deux sources de machines, l'historien et la télémétrie. Au Cours 03, les tables de l'ERP, des garanties, du référentiel, de la GMAO et de la météo entrent dans la couche `RAW`, la télémétrie aussi, et la documentation se dépose telle quelle dans le bronze.

5. Le stockage objet et le partitionnement

Un lac d'entreprise vit dans un stockage objet : Amazon S3, Azure Blob Storage, Google Cloud Storage.

Dans un stockage objet	Ce que ça change
pas de vrais dossiers, seulement des clés : `lac/bronze/usine/historien/jour=2025-10-28/data_0.parquet` est un nom	lister un « dossier » est une recherche par préfixe, d'autant plus chère qu'il y a d'objets
un objet ne se modifie pas : il se remplace en entier	on n'ajoute pas une ligne à un fichier ; on le réécrit, ou l'on en ajoute un autre
on paie à la requête et à l'octet transféré	lire moins de fichiers et moins d'octets économise de l'argent

- Le lac de ce parcours vit dans `plateforme/lac/`, sur votre disque, et se range comme s'il vivait dans un stockage objet.

© 2026 iAlexMG. Tous droits réservés.

- Le partitionnement par chemin met une valeur dans la clé : `jour=2025-10-28/`. C'est la convention de Hive, que DuckDB, Spark et Databricks reconnaissent : la partition devient une colonne, `jour`.
- L'élagage des partitions (*partition pruning*) : un filtre sur `jour` n'ouvre même pas les autres fichiers. Sur l'historien, le 28 octobre se lit dans un fichier sur 92.

Critère du choix de la clé	Ce qu'il demande	Chez Boréal
Le grain de livraison	ce qu'on remplacera d'un bloc quand une livraison est reprise	un jour de l'historien, un jour de réception de la télémétrie
Le filtre fréquent	ce par quoi les consommateurs filtrent	la date
La cardinalité	combien de partitions la clé produit	92 jours ; par étiquette et par jour, 5 336 partitions presque vides

- La partition suit le fichier livré, pas son contenu : les valeurs de départ du 30 septembre, en tête du fichier du 1er octobre, restent dans `jour=2025-10-01`. Au Cours 03, un jour relivré remplace sa partition, et rien d'autre.

Piège — les petits fichiers. Chaque fichier a un coût fixe : l'ouvrir, lire son pied de page, payer une requête. La télémétrie de Boréal fait 843 fichiers de 13 Ko en moyenne, et les lire tous prend plus de dix fois le temps de lire le même contenu en un seul fichier. Le remède, le compactage, réécrit les petits fichiers en quelques gros (Cours 10).

6. DuckDB sur le lac

Geste	Instruction
lire tout le bronze de l'historien	`FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet'` : la colonne `jour` apparaît d'elle-même
déclarer les types d'un CSV	`read_csv(…, columns = {'TagName': 'VARCHAR', …})`
déclarer la forme d'un JSON lines	`read_json(…, columns = {…})`, structures imbriquées comprises
écrire du Parquet partitionné	`COPY (…) TO 'dossier' (FORMAT parquet, COMPRESSION zstd, PARTITION_BY (jour), OVERWRITE)`
lire le schéma embarqué	`parquet_schema('fichier.parquet')`
lire les groupes de lignes et leurs statistiques	`parquet_metadata('fichier.parquet')`

© 2026 iAlexMG. Tous droits réservés.

Geste	Instruction
voir combien de fichiers une requête lit	`EXPLAIN ANALYZE SELECT …`

- `COPY` crée le dossier final, pas ses parents : `plateforme/lac/bronze/usine/` doit exister avant.
- `COPY` refuse un dossier qui n'est pas vide, sauf avec `OVERWRITE`, qui le vide d'abord. Relancer le script redonne le même bronze : un premier pas vers l'idempotence du Cours 03.
- La même requête sur l'historien : environ 3 s sur les CSV aux types devinés, 0,6 s aux types déclarés, moins de deux centièmes sur le bronze. Deviner coûte à chaque requête ; déclarer corrige à la fois la justesse et une bonne part de la lenteur ; le Parquet fait le reste.

7. Contrôler un bronze, en pratique

Un bronze écrit n'est pas un bronze livré. Avant de le déclarer bon, l'ingénieur de données le compare à sa source, avec les mêmes questions à chaque fois :

Question	Ce qu'on mesure	Chez Boréal (labo)
Tout est-il passé ?	les lignes de la source contre celles du bronze	275 436 et 158 146 des deux côtés (1.3, 2.2)
Chaque livraison est-elle entière ?	les lignes par partition, source contre bronze	92 et 843 jours, 0 écart (3.2, 3.7)
Les types sont-ils ceux qu'on a déclarés ?	le schéma du bronze	`DESCRIBE` (3.3, 3.8)
Les absences ont-elles survécu ?	les valeurs nulles des deux côtés	321 et 321 (3.4)
La partition suit-elle la livraison ?	ce qu'une partition contient hors de son jour	58 valeurs de départ dans `jour=2025-10-01` (3.5)
Le reçu est-il intact ?	ce que le bronze devait garder tel quel	deux noms de température, huit décalages horaires (3.9 à 3.11)
Se relance-t-il sans rien doubler ?	le contrôle après une seconde exécution	le même 275 436 (1.4)

Ces contrôles se font à la main au premier chargement. Le pipeline du Cours 05 les relance à chaque passage, et le Cours 11 en fait des contrats.

À retenir

- Un format en colonnes ne lit que les colonnes demandées et saute ce que ses statistiques excluent. Les applications transactionnelles restent en lignes.
- CSV et JSON servent à l'échange, Parquet au lac, Avro aux flux.

© 2026 iAlexMG. Tous droits réservés.

- Parquet porte son schéma et ses statistiques, et se compresse colonne par colonne : l'historien passe de 13,7 Mo à 1,5 Mo.
- Le bronze déclare ses types, et garde en texte les décalages horaires, les codes, les dates SAP. Un montant va en `DECIMAL`.
- Dans un lac, le chemin sert d'index : on partitionne par jour livré, on élague au filtre, et l'on se méfie des petits fichiers.
- `COPY` ne crée pas les dossiers parents, et refuse un dossier plein sans `OVERWRITE`.
- Un bronze se contrôle contre sa source, partition par partition, avant d'être livré.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_02_DE.md`. Faites-les après le labo : plusieurs énoncés supposent que le bronze existe.

Annexe A — Le détail des étapes du labo

A.1 Étape 1 : le bronze de l'historien

Labo : `Labo_02_DE.md`, étape 1.

Notions : section 4, la règle du bronze ; section 5, le partitionnement ; section 6, `COPY` et ses dossiers.

- 1.1. Le script est du code : il vit dans le dépôt, et se relancera à chaque livraison. Le Cours 05 en fera une pièce du pipeline. Ce que fait chaque morceau :
  - `columns = {…}` déclare le type de chaque colonne : plus de détection ;
  - `filename = true` ajoute le nom du fichier lu, d'où l'on tire le jour livré ;
  - `CAST(REPLACE(PARSE_FILENAME(…)))` fait de `historien_2025-10-28.csv` la date 2025-10-28 ;
  - `PARTITION_BY (jour)` écrit un dossier par jour, `jour=2025-10-28/` ;
  - `COMPRESSION zstd` choisit la compression du bronze (section 3) ;
  - `OVERWRITE` vide le dossier avant d'écrire ;
  - le dernier `SELECT` est le contrôle : les lignes lues contre les lignes écrites.

© 2026 iAlexMG. Tous droits réservés.

- 1.2. DuckDB crée le dossier final d'un `COPY`, `historien`, jamais ses parents. Le message finit par le texte de Windows, « Le chemin d'accès spécifié est introuvable ». Rien n'est écrit.
- 1.3. Sous Windows, `mkdir` crée les dossiers intermédiaires d'un coup, mais veut des `\` : avec `/`, il prend le chemin pour une option. Deux nombres égaux au contrôle : le bronze contient tout l'historien. Deux nombres différents diraient que les sources ont changé, ou que le script s'est arrêté en route.
- 1.4. `OVERWRITE` refait le dossier de zéro : relancer donne le même bronze et le même contrôle. Le Cours 03 fait de ce comportement une règle, l'idempotence.
- 1.5. Sans `OVERWRITE`, `COPY` refuse avant d'écrire : les 92 fichiers restent intacts (3.1 le confirme). La commande essaie d'écrire une ligne factice ; le refus est ce qu'on veut voir.

A.2 Étape 2 : le bronze de la télémétrie

Labo : `Labo_02_DE.md`, étape 2.

Notions : section 4, la règle du bronze ; section 6, déclarer la forme d'un JSON lines.

- 2.1. La forme du message est déclarée en entier : un `STRUCT` dans lequel `position` est une autre structure et `codes` une liste de textes. `recu_le`, `debut` et `fin` restent du texte, décalage compris. Les deux noms de la température, avant et depuis le micrologiciel 2.0, sont déclarés tous les deux : le dictionnaire de Boréal les donne.
- 2.2. Le contrôle lit l'archive en ne déclarant qu'une colonne : pour compter des lignes, c'est assez, et la détection n'a plus rien à rater (le piège de la colonne `JSON`, Labo 01, 4.6).

A.3 Étape 3 : contrôler chaque bronze contre sa source

Labo : `Labo_02_DE.md`, étape 3.

Notions : section 7, contrôler un bronze ; section 4, la règle du bronze ; section 5, la partition qui suit la livraison.

- 3.1. Les 13,7 Mo de CSV notés au Labo 01 (2.3) tiennent en 1,5 Mo, neuf fois moins, pour le même contenu.
- 3.2. Le contrôle du script compare des totaux ; celui-ci compare chaque jour. Un jour perdu et un jour doublé s'annuleraient dans un total, pas ici. `IS DISTINCT FROM` compte aussi un jour présent d'un seul côté, où l'un des deux comptes est nul. Le `FULL JOIN … USING (jour)` garde les jours des deux côtés, et `IS DISTINCT FROM` compare en comptant les `NULL` : les deux sont au Cours 01 de la formation SQL avancé, sections 7 et 8.
- 3.3. Les types sont ceux que le script a déclarés, plus `jour`, que DuckDB lit dans le chemin : aucune ligne ne l'écrit dans un fichier.
- 3.4. Les 321 lignes de qualité 0 arrivent sans valeur. Déclarée `DOUBLE`, la colonne garde ces vides en nul : la mesure manquante reste une information.
- 3.5. Le fichier du 1er octobre commence par la dernière valeur de chacune des 58 étiquettes le 30 septembre. Elles restent dans le jour qui les a livrées : la partition suit la livraison (section 5).

© 2026 iAlexMG. Tous droits réservés.

- 3.6. Les 70,6 Mo de JSON lines tiennent en 10,8 Mo. Les 41 jours sans message ne donnent aucun dossier : `COPY` ne partitionne que ce qui existe, d'où 843 fichiers pour 884 jours.
- 3.7. Le même contrôle, jour par jour. Les 41 fichiers vides ne donnent aucune ligne, ni d'un côté ni de l'autre.
- 3.8. La forme déclarée en 2.1, intacte : `message` est une seule colonne, structurée.
- 3.9. `COUNT` d'une colonne ne compte que les valeurs présentes : chaque message remplit l'un des deux champs, jamais les deux. Les réunir est une transformation, qui appartient à l'ingénieur analytique ; le bronze les garde tels que reçus.
- 3.10. `recu_le` et `debut` gardent leur décalage, `-05:00`. Au Labo 01 (4.4), le même `recu_le`, deviné, avait perdu le sien.
- 3.11. `debut` et `fin` sont à l'heure du véhicule. Huit décalages, de `-02:30` (Terre-Neuve, à l'heure avancée) à `-08:00` (Colombie-Britannique, à l'heure normale) : ramenés à UTC, rien ne dirait plus dans quel fuseau roulait chaque véhicule.

A.4 Étape 4 : ce que le Parquet sait de lui-même

Labo : `Labo_02_DE.md`, étape 4.

Notions : section 3, Parquet de près ; section 5, le piège des petits fichiers.

- 4.1. Un horodatage est un entier de 64 bits, compté en microsecondes ; une chaîne, une suite d'octets marquée UTF-8. Ces types voyagent avec le fichier : aucun outil n'a plus à les deviner.
- 4.2. Trois lectures du pied de page :
  - le fichier du 28 octobre commence le 27 à minuit : les valeurs de départ ;
  - `TagName` couvre toutes les étiquettes, de la première à la dernière : un filtre sur une étiquette ne sautera rien dans ce fichier (section 3) ;
  - `Value`, des nombres qui varient sans cesse, pèse 12 598 octets compressés ; `Quality`, presque toujours 192, tient en 94.
- 4.3. Des fichiers de 3 à 48 Ko : chacun répète son pied de page, son schéma et ses dictionnaires. C'est le piège des petits fichiers (section 5), mesuré en 5.3.

A.5 Étape 5 : mesurer ce que le format change

Labo : `Labo_02_DE.md`, étape 5.

Notions : section 1, ligne contre colonne ; section 3, la projection et le prédicat poussé ; section 5, l'élagage et les petits fichiers ; section 6, DuckDB sur le lac.

- 5.1. Trois écarts, trois causes :
  - de 3 s à 0,6 s : la détection des types, refaite à chaque requête ;

© 2026 iAlexMG. Tous droits réservés.

    - de 0,6 s à 0,02 s : le format ; le CSV oblige à lire chaque octet et à convertir chaque valeur, le Parquet ne lit que deux colonnes, déjà typées et compressées ;
    - vos temps seront autres ; l'ordre de grandeur, non.

- 5.2. `EXPLAIN ANALYZE` exécute la requête et montre ce qu'elle a lu. `Scanning Files: 1/92` : le filtre portait sur la partition, donc sur le chemin, et DuckDB n'a ouvert qu'un fichier. Un filtre sur `Timestamp` en lirait 92, pour ouvrir chaque pied de page (exercice P3).
- 5.3. Le bronze gagne encore, mais bien moins que sur l'historien : ouvrir 843 fichiers de 13 Ko coûte plus que les lire. L'exercice P5 mesure ce que rendrait un seul fichier.

A.6 Étape 6 : versionner les scripts, jamais le lac

Labo : `Labo_02_DE.md`, étape 6.

Notions : aucune section de ce cours ; le dépôt `01_de` et ce qu'il garde viennent du Cours 01, annexe A.1.

- Les scripts vont dans le dépôt ; le bronze vit dans `plateforme/`, hors du dépôt, et se reconstruit en les relançant.
- `git add` nomme les deux fichiers : rien d'autre n'entre dans le commit par mégarde.

En entreprise, le bronze vit dans le stockage de la plateforme, et les mêmes gestes changent d'outil :

Geste du labo	**Snowflake**	**Databricks**
Écrire le bronze (1, 2)	`COPY INTO` une table, depuis un stage	Auto Loader ou `COPY INTO` une table Delta : des fichiers Parquet et un journal de transactions
Partitionner et élaguer (3, 5.2)	automatique : des micro-partitions de 50 à 500 Mo non compressés, et une clé de regroupement (*clustering key*) au besoin ; le profil de requête montre les partitions lues sur le total	les statistiques de chaque fichier ; Databricks recommande le regroupement dynamique (*liquid clustering*) plutôt que le partitionnement par chemin
Lire le schéma d'un fichier (4.1)	`INFER_SCHEMA` sur un stage	`DESCRIBE QUERY SELECT * FROM read_files('s3://…/historien/', format => 'parquet')`
Compacter (5.3)	rien à faire : le service s'en charge	`OPTIMIZE`, que l'optimisation prédictive lance d'elle-même sur les tables gérées par Unity Catalog (Cours 10)

© 2026 iAlexMG. Tous droits réservés.